

MoMo SMS Transactions REST API

Implementation, Security, Documentation and DSA Analysis

Team name: Triple Threat

Team members: Ivan Ineza Hakizimana, Samuel Hezekiah Epodoi, Devis Muhozi

GitHub repository: momo-transaction-analytics

Course: Enterprise Web Development, African Leadership University

1. Introduction to API Security

An API exposes data and operations to any client that can reach it, which makes it the main attack surface of a system. The MoMo dataset contains financial information (amounts, balances, counterparties and transaction ids). Serving it without controls would leak private data and allow records to be tampered with.

API security is built around three questions:

Authentication asks who is calling. This project uses HTTP Basic Authentication: the client sends an `Authorization: Basic base64(user:password)` header on every request.

Authorization asks what the caller may do. Here one account can do everything; a production system would separate read-only and write roles.

Integrity and input validation asks whether the request is safe to process. Every POST and PUT body is checked for valid JSON, required fields, field types, non-negative amounts, timestamp format and unknown fields. Failures return 400.

Controls implemented in api/server.py

Control	Implementation
Authentication on every endpoint	Checked before any route runs. Missing or invalid credentials return 401 with a WWW-Authenticate header.
Credentials kept out of the code	The username and password are read from the API_USER and API_PASSWORD environment variables. The server refuses to start without them.
Constant-time comparison	The password is compared with <code>hmac.compare_digest</code> , which avoids timing differences that could leak information.
Input validation	JSON shape, required fields, types, non-negative amount, timestamp format (YYYY-MM-DD HH:MM:SS) and unknown fields are checked. Failures return 400 with a reason.
Method restrictions	405 when a method is used on the wrong route, for example POST /transactions/5.

Data parsing

`dsa/parse_xml.py` reads the SMS backup with `ElementTree`. The useful information is inside the free-text *body* attribute, so regular expressions classify each message into one of 12 types (`incoming_money`, `payment`, `transfer`, `bank_deposit`, `withdrawal`, `airtime`, `cash_power`, `bundle_purchase`, `third_party_payment`, `reversal`, `failed_transaction` and `otp`) and extract the amount, sender, receiver, balance, transaction id and timestamp. The file contains 1,691 messages (the XML header claims 1,693), and each becomes a dictionary with a sequential id from 1 to 1691. Messages with no time in the text use the SMS date attribute converted to Rwanda time (UTC+2). The 8 OTP messages have no amount, sender or receiver.

2. Documentation of Endpoints

Base URL: `http://127.0.0.1:8000`. All endpoints require Basic Auth and return JSON. Full documentation with examples is in `docs/api_docs.md`.

Method and path	Purpose	Success	Errors
GET /transactions	List all transactions	200	401

GET /transactions/{id}	Get one transaction	200	401, 404
POST /transactions	Create a transaction	201 + Location header	400, 401
PUT /transactions/{id}	Update the fields sent in the body	200	400, 401, 404
DELETE /transactions/{id}	Delete a transaction	200	401, 404

Request and response example (POST)

```
curl -u "$API_USER:$API_PASSWORD" -X POST http://127.0.0.1:8000/transactions \
-H "Content-Type: application/json" \
-d '{"type":"transfer","amount":3000,"sender":"You",
    "receiver":"Grace Uwase","timestamp":"2025-02-01 10:15:00",
    "balance":47000}'
```

```
HTTP/1.0 201 Created
Location: /transactions/1692
```

```
{"id": 1692, "type": "transfer", "amount": 3000, "sender": "You",
  "receiver": "Grace Uwase", "timestamp": "2025-02-01 10:15:00",
  "balance": 47000, "transaction_id": null, "raw_body": null}
```

Error codes

Code	Meaning
200	OK
201	Created
400	Malformed JSON, missing or invalid field, unknown field, bad timestamp, negative amount
401	Missing or wrong credentials
404	Unknown route or transaction id
405	Method not allowed on this route

Test results (curl)

The full run is in docs/curl_test_output.txt and the automated tests are in tests/test_api.py. Screenshots are in the screenshots/ folder. Observed status codes:

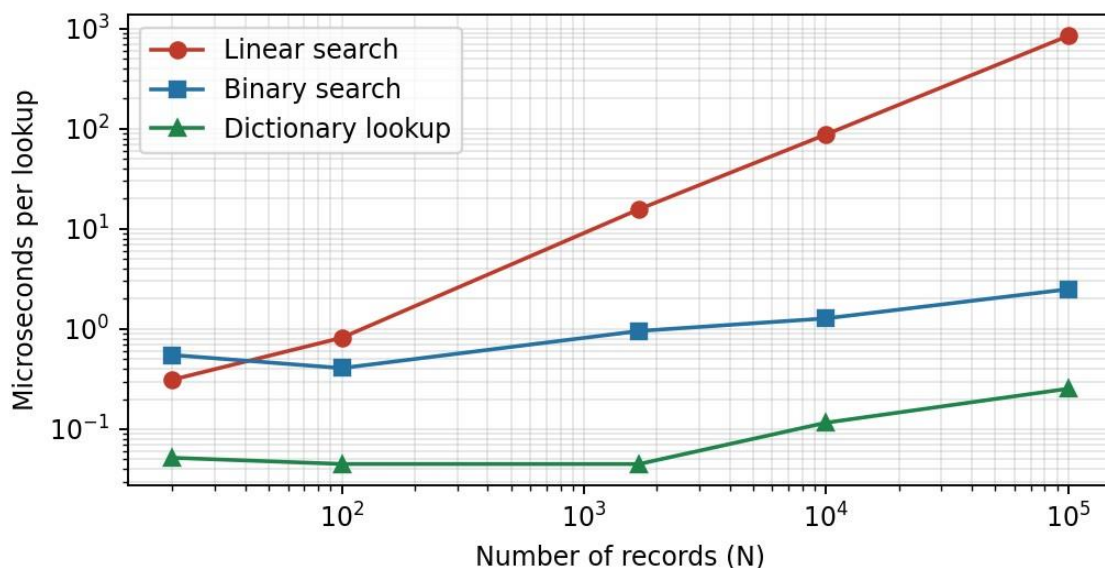
Test	Result
GET /transactions and /transactions/1 with correct credentials	200 OK (list of 1,691 records, or one record)
GET with a wrong password or no login	401 Unauthorized with WWW-Authenticate header
POST with a valid body	201 Created, id 1692 assigned
PUT /transactions/1692 (amount, balance)	200 OK, fields updated
DELETE /transactions/1692	200 OK; a later GET returns 404
POST with invalid data or a bad timestamp	400 Bad Request with the reason

3. Results of the DSA Comparison

Two ways of finding a transaction by id were implemented in `dsa/search_compare.py`. **Linear search** scans the list from the start until the id matches, which is $O(n)$. **Dictionary lookup** uses a dict of id to transaction, which is $O(1)$ on average. Binary search on the sorted id list ($O(\log n)$) was added as an extra.

For each dataset the script searches for up to 200 random ids, repeated 20 times (2 times for the largest lists), and averages the time per search with `time.perf_counter`. The first three rows use the real parsed XML records (20, 100 and all 1,691, which covers the required minimum of 20). The last two rows use generated lists to show what happens as data grows. The script also checks that all three methods return the same record.

Dataset	N	Linear (us)	Dictionary (us)	Binary (us)	Linear / Dict
first 20 records	20	0.312	0.052	0.550	6x
first 100 records	100	0.818	0.045	0.407	18x
all XML records	1,691	15.706	0.045	0.955	350x
fake data	10,000	87.597	0.116	1.277	755x
fake data	100,000	844.423	0.253	2.495	3331x



Reflection

Why is dictionary lookup faster? A Python dict is a hash table. The key goes through a hash function that maps it straight to a slot, so the position of the record is computed rather than searched for. The cost stays roughly the same as N grows, while linear search inspects about $n/2$ elements on average. The measurements agree: the dictionary stays between about 0.05 and 0.3 microseconds at every size, while linear search grows roughly tenfold for every tenfold increase in records.

Trade-offs. The dictionary uses extra memory for the hash table and must stay in sync when records are added or deleted (the API updates it directly). With only 20 records the difference is a fraction of a

microsecond, so the real benefit is scalability. Binary search was slower than linear search at 20 records because of its loop overhead, and only won as the list grew.

Other structures that could improve search. A sorted list with binary search gives $O(\log n)$ and supports range queries, but insertions cost $O(n)$. A balanced binary search tree or B-tree keeps data sorted with $O(\log n)$ inserts, deletes and range scans, for example all transactions between two dates. Secondary indexes, such as a dict from transaction type to a list of ids, would speed up filtering by fields other than id. A trie would support prefix search on names. For large persistent data, a database index (B-tree), like the ones in this project's MySQL design, is the practical choice.

4. Reflection on Basic Auth Limitations

Basic Authentication is simple and universally supported, which makes it good for a learning exercise, but it is weak for real deployments.

Weakness	Why it matters
Credentials are encoded, not encrypted	Base64 is trivially reversible. Anyone who sees the header over plain HTTP (public Wi-Fi, a proxy, a log) can read the password.
Password sent on every request	The long-lived secret is exposed many times, raising the chance of leaks through logs, browser history or proxies.
No expiry or revocation	A stolen password works until it is changed. One device or session cannot be revoked.
No scopes or roles	The same credential grants all access; a partner app cannot be given read-only access.
No brute-force protection by design	Without rate limiting or lockout, passwords can be guessed repeatedly.
Poor fit for browsers and mobile apps	Apps must store the raw password, and there is no logout or built-in multi-factor support.
Shared static credentials	Hard to rotate or audit; you cannot tell which client performed an action.

Stronger alternatives

JWT (JSON Web Tokens). The user logs in once (POST /login) and receives a signed, short-lived token with claims such as user id, role and expiry. Clients send `Authorization: Bearer <token>`. The server verifies the signature without a database lookup, the password is not re-sent, tokens expire automatically and roles can be embedded. Risks to manage: keep expiry short, use refresh tokens, protect the signing key, and remember that a stateless token cannot be revoked before it expires unless a deny-list is kept.

OAuth 2.0 (with OpenID Connect). A delegation framework in which a third-party app gets a scoped access token from an authorization server after the user consents, without ever seeing the password. It supports scopes (for example transactions:read), expiry, revocation, refresh tokens and integration with identity providers and multi-factor authentication. It is the standard for public APIs and for mobile money ecosystems with many partner apps.

Other improvements. Always serve over HTTPS/TLS; store passwords hashed (bcrypt or argon2) rather than in plain configuration; add rate limiting and account lockout; use API keys or mutual TLS for service-to-service calls; add audit logging and multi-factor authentication for sensitive operations.

Conclusion

The API meets the assignment goals: full CRUD, protection of every endpoint, documented behaviour and a measured search-efficiency comparison. For production use with financial data it should move to HTTPS and token-based authentication (JWT or OAuth 2.0) backed by a database, such as the MySQL design from Week 2.